



VULDA: Source Code Vulnerability Detection via Local Dependency Context Aggregation on Vulnerability-Aware Code Mapping Graph

Tao Peng¹, Ling Gui¹, Lijun Cai^{2(✉)}, Junwei Tang¹, Aoshuang Ye¹,
and Fei Zhu¹

¹ School of Computer Science and Artificial Intelligence, Wuhan Textile University,
Wuhan 430063, China

² College of Computer Science and Electronic Engineering, Hunan University,
Changsha, China

{pt,2315363137,jwtang,asye}@wtu.edu.cn, ljcai@hnu.edu.cn

<https://github.com/guilingxz/VULDA>

Abstract. Vulnerability detection is crucial in the field of software security. However, existing methods often suffer from interference caused by redundant information and insufficient cross-line semantic dependencies when handling large-scale and complex source code, which limits detection performance. To address these challenges, this paper proposes VULDA, a source code vulnerability detection method that integrates a Vulnerability-aware Code Mapping Graph (VCMG) with Local Dependency Context Aggregation (LDCA). VCMG significantly reduces redundancy in graph structures by aligning multi-granularity semantics to line-level nodes, thereby enhancing representational compactness. Additionally, it incorporates static heuristic rules and structural features to weight nodes, effectively improving the model's sensitivity to key vulnerability-related code. Building upon this, the LDCA module aggregates both control-flow graph (CFG) and data-dependency graph (DDG) paths, achieving dual-context aggregation of logical and data semantics, which further enhances the model's ability to express complex vulnerability patterns. Experimental results on multiple real-world datasets, including SARD, Reveal, and FFmpeg+Qemu, demonstrate that VULDA outperforms existing methods across various metrics, notably achieving a 23.09% improvement in F1 score on the Reveal dataset compared to the best baseline. Ablation studies further validate the effectiveness and complementarity of the VCMG and LDCA modules in boosting detection performance.

Keywords: Vulnerability Detection · Software Security · Code Semantics · Vulnerability Patterns · Deep Learning

1 Introduction

With the widespread use of information technology, software systems have become vital in sectors such as finance, healthcare, and transportation, where their security directly affects societal stability. Vulnerabilities in source code serve as major entry points for cyberattacks, causing serious consequences like data breaches and service disruptions [1, 2]. Efficient and accurate detection of such vulnerabilities is thus a core concern in software security [3].

Traditional static analysis relies on manual rules or similarity matching, which often struggle with unknown vulnerabilities and complex semantic dependencies [4, 5]. Deep learning methods have recently advanced vulnerability detection by representing source code as graphs, natural language, or images, enabling automatic feature learning. However, two key challenges persist: vulnerability code is rare in real projects, with abundant redundant lines interfering with training [6, 7]; and vulnerabilities often involve cross-line and cross-path semantics, which single-line or shallow models fail to capture [3].

Previous work like Devign [3] and AMPLE [8] reduce irrelevant information via node selection, graph modeling, or program slicing. However, AST-based graphs still suffer from redundancy and semantic loss [7], and slicing may omit critical semantics [9]. These methods struggle to balance compactness with contextual completeness, limiting their performance in complex scenarios [10].

To address these issues, we propose **VULDA**, a deep learning framework that constructs a *Vulnerability-aware Code Mapping Graph (VCMG)* to compress code structure and integrate static heuristic features, combined with *Local Dependency Context Aggregation (LDCA)* to fuse control-flow and data-flow semantics for precise vulnerability context modeling. Experiments on multiple real datasets show significant performance improvements.

Our main contributions are:

1. A novel framework, **VULDA**, that integrates structural optimization and context aggregation to improve vulnerability detection in complex code.
2. The design of **VCMG**, which models line-level nodes with semantic weighting, reducing redundancy and enhancing key code representations.
3. The **LDCA** method that aggregates control-flow and data-flow paths to capture complex cross-line semantics, enhancing contextual modeling.

2 Background and Related Work

With the continuous growth in software scale and complexity, vulnerability detection techniques have rapidly evolved from rule-based methods and traditional machine learning to deep learning approaches. Early methods relied on expert-crafted rules, such as Checkmarx [11], FlawFinder [12], and RATS [13], which efficiently detected specific vulnerabilities but struggled to adapt to unknown ones and incurred high maintenance costs. Subsequently, traditional

machine learning techniques were introduced, leveraging static or dynamic features (e.g., code complexity [1, 14, 15], change frequency [14, 16, 17], and test coverage [18, 19]) to train classifiers, improving automation to some extent. However, these approaches depended heavily on manual feature engineering, limiting their ability to capture deep semantic relationships and resulting in modest detection performance [20].

Recently, deep learning has made significant progress in vulnerability detection due to its powerful feature representation capabilities. Sequence-based models, such as VulDeePecker [2], SySeVR [9], and VulDeeLocator [21], use RNNs to model program slices, reducing manual intervention. However, slicing often leads to loss of contextual semantics, making it difficult to capture cross-line or cross-function dependencies. Abstract Syntax Tree (AST)-based methods capture some structural information but have limitations in modeling control-flow and data dependencies [7]. Graph Neural Network (GNN) methods, including Devign [3], GRACE [22], and AMPLE [8], improve vulnerability representation by constructing code graphs and have shown promising results. Yet, they still face challenges related to graph structure redundancy and insufficient modeling of cross-line semantics [6, 23].

To overcome these limitations, we design **VULDA** as a unified framework that synergizes structural compression and contextual semantics aggregation. The technical details of its core components (VCMG and LDCA) will be elaborated in Sect. 3.

3 System Architecture

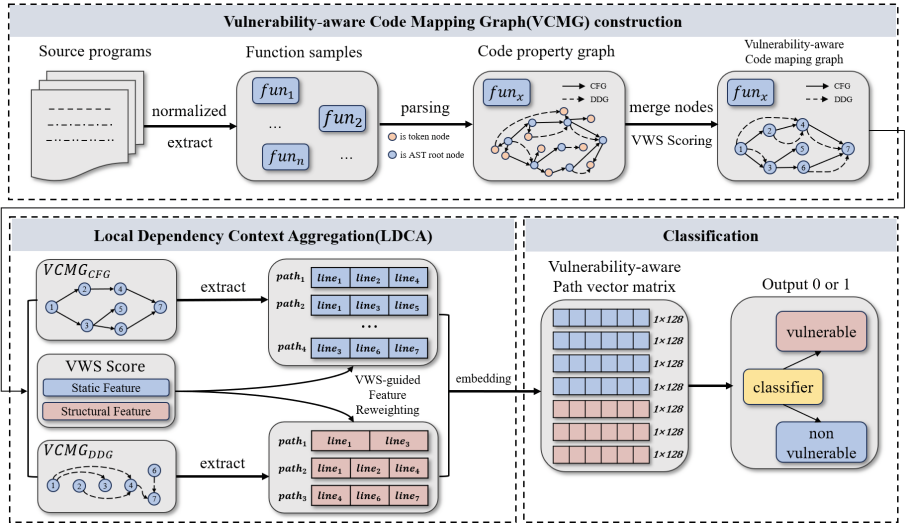


Fig. 1. Overview of the VULDA system architecture.

3.1 Overview

As illustrated in Fig. 1, the overall **VULDA** system consists of three stages: Vulnerability-aware Code Mapping Graph (VCMG) construction, Local Dependency Context Aggregation (LDCA), and classification.

- **VCMG Construction:** Code Property Graphs (CPGs) are built at the function level and mapped into structures based on line-level code nodes. This graph integrates static features and structural features, and incorporates a vulnerability-aware scoring mechanism to weight each node, thereby compressing the graph structure while enhancing key semantic representations.
- **LDCA Extraction:** Local paths related to potential vulnerabilities are extracted from the Control Flow Graph (CFG) and Data Dependency Graph (DDG), and weighted aggregation is performed using node vulnerability scores. This extraction effectively captures complex cross-line and cross-path dependencies, improving contextual modeling capabilities.
- **Classification:** The aggregated path representations are encoded as vector matrices via *Sent2Vec* [24] and fed into a deep learning classifier, which outputs the final vulnerability detection results.

3.2 Construction of Vulnerability-Aware Code Mapping Graph

The construction of VCMG involves three stages. First, function-level code slices are extracted from the source code and normalized (e.g., standardized naming, comment removal) to reduce semantic noise caused by naming variations, enabling the model to focus on vulnerability-related structures and patterns (Fig. 2(a)→(b)). Second, each function is converted into a Code Property Graph (CPG) using the Joern tool [25], capturing multi-dimensional relations such as syntax, control flow, and data flow to form an initial graph structure (Fig. 2(c)). Third, to reduce node redundancy and weak semantic noise caused by AST parsing, token nodes are aggregated at the code line level while preserving original dependency edges, resulting in a more compact graph. Meanwhile, a vulnerability weight score (VWS), computed based on static heuristic rules, is embedded into each node to enhance the representation of critical code regions. The resulting VCMG effectively highlights potential vulnerability areas while preserving structural semantic integrity (Fig. 2(d)). The detailed design of VWS is presented in Sect. 3.3.

3.3 Vulnerability Weight Score (VWS)

To quantify the importance of each source code line with respect to vulnerabilities, we design a *Vulnerability Weight Score (VWS)* that combines static semantic features and structural centrality metrics to guide the weighted aggregation of semantic paths.

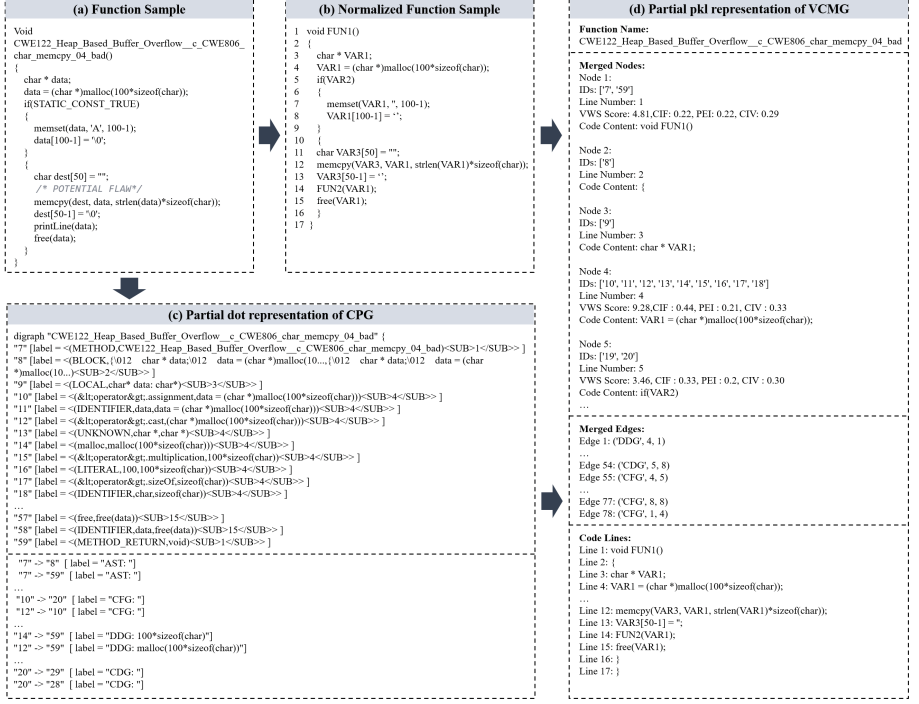


Fig. 2. Illustration of the VCMG generation process based on a buffer overflow vulnerability example.

The VWS is calculated as follows:

$$\text{VWS} = \text{FC} + \text{AU} + \text{PU} + \text{AE} + k \times C \quad (1)$$

where FC denotes the frequency of function calls, reflecting external interface activity; AU represents array usage frequency, often related to out-of-bounds vulnerabilities; PU indicates pointer usage, which may cause illegal memory access; AE captures the complexity of arithmetic expressions, associated with overflow risks; and C is the structural centrality score of the code line in the code mapping graph. To emphasize structural features, C is amplified by a factor $k = 10$.

The structural centrality score C integrates three classical graph centrality metrics:

Degree Centrality (C_D):

$$C_D(v) = \frac{\deg(v)}{N - 1} \quad (2)$$

Betweenness Centrality (C_B):

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (3)$$

Clustering Coefficient (C_L):

$$C_L(v) = \frac{2 \times \text{Number of links between neighbors of } v}{\deg(v) \times (\deg(v) - 1)} \quad (4)$$

The overall structural centrality C is computed as a weighted sum:

$$C = \omega_1 \times C_D + \omega_2 \times C_B + \omega_3 \times C_L \quad (5)$$

with weights $\omega_1 = 0.4$, $\omega_2 = 0.3$, and $\omega_3 = 0.3$.

By unifying static and structural features, VWS enables fine-grained differentiation of vulnerability-relevant code lines. Lines with higher scores receive greater weights in the subsequent LDCA aggregation, enhancing detection performance.

3.4 Local Dependency Context Aggregation (LDCA)

Vulnerabilities often span multiple lines of code and depend on complex control flow and data dependencies, making single-line analysis prone to missing critical patterns. To address this, we propose the Local Dependency Context Aggregation (LDCA) method, which extracts control flow (CFG) and data dependency (DDG) paths from the Vulnerability-aware Code Mapping Graph (VCMG) and performs weighted aggregation guided by the Vulnerability Weight Score (VWS) to enhance vulnerability semantic representation.

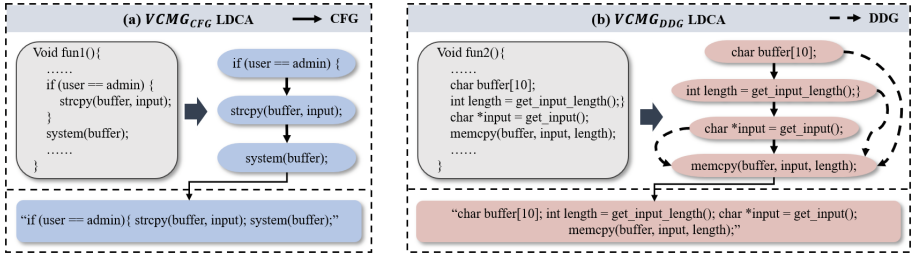


Fig. 3. Illustration of Local Dependency Context Aggregation (LDCA) Process: (a) Control Flow Context Aggregation, (b) Data Dependence Context Aggregation.

LDCA focuses on two dimensions: control flow context and data dependency context. Control flow aggregation (CFG) preserves the relationship between conditional checks and subsequent execution paths, while data dependency aggregation (DDG) traces variable origins and usages to uncover potential data-driven vulnerabilities.

Specifically, the examples in Fig. 3 illustrate the two aggregation processes. Figure 3(a) shows how the condition `if (user == admin)` controls the execution of `strcpy(buffer, input);` and `system(buffer);`. Ignoring this condition would result in missed vulnerabilities. LDCA extracts continuous control flow paths to reconstruct the complete control context. Figure 3(b) demonstrates the variable `length` propagated from `get_input_length()` to `memcpy`. Without tracking this source, buffer overflow risks may be overlooked. LDCA traces DDG paths to fuse related code segments, providing a complete data context.

Algorithm 1. Local Dependency Context Aggregation (LDCA)

Require: Code list `code_list`, edge list `merged_edges`, graph types CFG and DDG
Ensure: Aggregated path list `fused_paths`

```

1: Initialize fused_paths  $\leftarrow []$ 
2: Initialize cfg_paths  $\leftarrow \emptyset$ 
3: Initialize ddg_nodes  $\leftarrow \{\}$ 
   {Single pass: Process CFG and DDG edges together}
4: for each  $(\text{graph\_type}, \text{source\_line}, \text{target\_line}) \in \text{merged\_edges}$  do
5:   if graph_type = CFG then
6:     Add  $(\text{source\_line}, \text{target\_line})$  to cfg_paths
7:   else if graph_type = DDG and source_line  $\neq 1$  and target_line  $\neq 1$  then
8:     if target_line  $\notin \text{ddg\_nodes}$  then
9:       ddg_nodes[target_line]  $\leftarrow []$ 
10:    end if
11:    Append code_list[source_line - 1] to ddg_nodes[target_line]
12:  end if
13: end for
   {Merge sequences of three consecutive CFG nodes}
14: for each  $(a, b) \in \text{cfg\_paths}$  do
15:   for each  $(b', c) \in \text{cfg\_paths}$  where  $b' = b$  do
16:     merged_code  $\leftarrow \text{code\_list}[a - 1] + \text{code\_list}[b - 1] + \text{code\_list}[c - 1]$ 
17:     Append merged_code to fused_paths
18:   end for
19: end for
   {Merge DDG dependencies for each target node}
20: for each target_line  $\in \text{ddg\_nodes}$  do
21:   merged_code  $\leftarrow \text{concatenate}(\text{ddg\_nodes}[\text{target\_line}]) + \text{code\_list}[\text{target\_line} - 1]$ 
22:   Append merged_code to fused_paths
23: end for
24: return fused_paths

```

The detailed steps of the LDCA algorithm are presented in Algorithm 1, which leverages CFG and DDG edge information to extract and integrate control flow and data dependency paths, generating localized contexts for improved vulnerability detection.

3.5 Classification

After Local Dependency Context Aggregation (LDCA), two types of fused local code snippets are obtained. Each context snippet is embedded into a fixed-length 128-dimensional vector using Sent2Vec [24]. These embeddings are then sequentially arranged in execution order to form a feature matrix as input to the classification model.

To enhance vulnerability feature representation, we apply a Vulnerability Weight Score (VWS)-based weighting strategy. Specifically, for each code snippet, the set of involved line numbers is extracted, and their corresponding VWS values are retrieved from the mapping (`vws_c_map`). The average score is computed and normalized via a soft attention mechanism to generate weights. The original vectors are then scaled by these weights, strengthening the model's focus on critical vulnerable code lines and improving detection performance.

To evaluate the classification performance of VULDA, we compare five representative deep learning models. The LSTM (Long Short-Term Memory) [26] model adopts a standard single-layer unidirectional architecture with a hidden size of 64. The BiLSTM (Bidirectional LSTM) [27] enhances sequential modeling by incorporating forward and backward contexts, also with a hidden size of 64. The GRU (Gated Recurrent Unit) [28] applies a simplified gating mechanism to improve training efficiency, using a hidden size of 64. The BiGRU (Bidirectional GRU) [29] combines bidirectional processing with a hidden size of 64×2 to improve long-sequence modeling. Finally, the CNN (Convolutional Neural Network) [30] employs one-dimensional convolutional kernels of sizes 3, 5, and 7 to extract features, followed by a fully connected layer for final classification.

4 Experiments

4.1 Research Questions

This section aims to address the following research questions:

- **RQ1:** How does VULDA perform across different classification models?
- **RQ2:** What is the individual contribution of each component within the VULDA framework to the overall performance?
- **RQ3:** How does VULDA compare with other state-of-the-art vulnerability detection methods?

Table 1. Statistics of the datasets

Dataset	Samples	Vul	Non-vul	Vul Ratio(%)
SARD	16,776	12,116	4,660	72.22
Reveal	18,169	1,664	16,505	9.16
FFmpeg+Qemu	22,361	10,067	12,294	45.02

4.2 Datasets

To evaluate the effectiveness of VULDA, we utilize three vulnerability datasets: SARD [31], Reveal [32], and FFmpeg+Qemu [3]. The statistical details of these datasets are presented in Table 1.

- **SARD Dataset** [31]: Sourced from SySeVR, this dataset encompasses 126 types of vulnerabilities, each uniquely identified using the Common Weakness Enumeration (CWE) ID. It includes data from production environments, synthetic sources, and academic test cases, categorized into three groups: *Good* (non-vulnerable), *Bad* (vulnerable), and *Mixed* (containing both vulnerabilities and patches). In total, the dataset comprises approximately 12k vulnerable samples and 4k non-vulnerable samples.
- **Reveal Dataset** [32]: Collected from two open-source projects, Linux Debian Kernel and Chromium, this dataset contains around 2k vulnerable samples and 20k non-vulnerable samples.
- **FFmpeg+Qemu Dataset**: Provided by Deign [3], this dataset consists of approximately 10k vulnerable samples and 12k non-vulnerable samples extracted from two open-source C projects. All samples have been manually labeled.

4.3 Baseline Methods

To evaluate the performance of VULDA, we compare it with the following baseline methods:

- **VulDeePecker** [2]: Utilizes program slicing techniques to convert code fragments into vector representations, employing a BLSTM model for vulnerability detection.
- **SySeVR** [9]: Extracts code features through syntactic analysis and semantic slicing, converting them into fixed-length vectors and training a BGRU model for detection.
- **Devign** [3]: Leverages a graph neural network (GNN) to encode function-level code as a joint graph structure. It applies gated graph recurrent layers and convolutional modules to learn node features for vulnerability detection.
- **VulCNN** [33]: Converts function source code into images while preserving program semantics, enabling efficient large-scale vulnerability detection.

- **GRACE** [22]: Integrates large language models (LLMs), graph structural information, and context learning, enhancing detection accuracy through demonstration-based retrieval techniques.

4.4 Experimental Setup and Environment

The dataset was randomly split into training, validation, and test sets in an 8:1:1 ratio to ensure independence and representativeness across the different stages of model development [34]. To standardize input formats and handle variable-length sequences, we applied padding and truncation: sequences longer than 200 tokens were truncated to the first 200 tokens, while shorter sequences were zero-padded at the end to maintain a uniform input length of 200 [35]. For data formatting, the processed samples were wrapped using the PyTorch `Dataset` class and loaded in batches of size 32 using the `DataLoader` utility [36].

During training, we used the cross-entropy loss function and the Adam optimizer [37], with the learning rate set to 0.001 and the number of training epochs fixed at 10. Detailed hardware and software configurations are summarized in Table 2. Model performance was comprehensively evaluated using standard classification metrics including Accuracy, Precision, Recall, and F1-Score.

Table 2. Details of the experimental devices

Device	Type	Version
GPU	Nvidia	RTX4090
CPU	Intel Xeon Platinum	8352V
Operating system	Centos linux release	7.9.2009
Package	Python	3.8
	Torch	1.12.1
	Matplotlib	3.8.2
	Numpy	1.25.2
	Networkx	3.1
	Joern	2.0.121

5 Experimental Results

5.1 Performance Across Different Classifiers

To evaluate the adaptability of various classifiers for the vulnerability detection task, we integrate five mainstream deep learning models—LSTM, BiLSTM, GRU, BiGRU, and CNN—into the complete VULDA framework. Classification experiments are conducted using path-level features extracted from three public

datasets: SARD, Reveal, and FFmpeg+Qemu. The evaluation metrics include Accuracy, Precision, Recall, and F1-Score.

As shown in Table 3, the results demonstrate significant performance variations across classifiers and datasets, reflecting differences in structural complexity and semantic characteristics of the source code. On the SARD dataset, LSTM achieves the best performance with an F1 score of 96.69, indicating its effectiveness in modeling relatively stable and semantically clear sequential data without the need for complex architectural designs. For the Reveal dataset, GRU performs best with an F1 score of 66.22, suggesting that its simplified gating mechanism strikes a good balance in handling intricate contexts and nested dependencies. On the FFmpeg+Qemu dataset, CNN leads with an F1 score of 55.55, highlighting its advantage in capturing local patterns and processing high-dimensional, heterogeneous features.

Based on these findings, the optimal classifier for each dataset is selected in subsequent experiments to ensure both stability and fairness in performance comparison.

Table 3. Performance Comparison of VULDA with Different Classifiers on Vulnerability Detection

Dataset	Methods	Accuracy	Precision	Recall	F1 score
SARD	LSTM	97.29	95.79	97.73	96.69
	BiLSTM	90.76	87.49	93.28	89.4
	GRU	97.08	95.64	97.32	96.43
	BiGRU	96.81	95.26	97.07	96.11
	CNN	96.96	95.37	97.37	96.3
Reveal	LSTM	90.7	71.74	59.72	62.69
	BiLSTM	90.55	70.93	61.03	63.91
	GRU	91.94	81.25	61.91	66.22
	BiGRU	91.4	77.16	60.22	63.84
	CNN	90.86	72.72	61.1	64.27
FFmpeg+Qemu	LSTM	51.53	48.94	49.29	45.61
	BiLSTM	52.76	48.66	49.58	40.52
	GRU	50.6	50.1	50.1	50.07
	BiGRU	53.1	49	49.76	39.45
	CNN	58.84	59.03	57.1	55.55

5.2 Ablation Study of VULDA Modules

To systematically evaluate the contribution of key components within the VULDA framework, we conduct a series of ablation experiments targeting the

Local Dependency Context Aggregation (LDCA) module—specifically its Control Flow Graph (CFG) and Data Dependence Graph (DDG) submodules—as well as the Vulnerability Weight Score (VWS) module. These experiments aim to quantify the performance gains attributable to each submodule and highlight their individual impact on the overall detection effectiveness.

Ablation of the LDCA Module. The LDCA module leverages VCMG’s multi-dimensional contextual information to enrich code semantic representation. In its full configuration, LDCA incorporates both control flow context from CFG and data dependency context from DDG. To assess the independent contributions of these components, ablation variants were constructed by selectively removing either CFG or DDG. The configurations of the LDCA ablation experiments are summarized in Table 4, and the experimental results for each configuration are reported in Table 5.

Table 4. Ablation Settings for Evaluating VCMG and LDCA Modules

Exp Name	VCMG	CFG	DDG	Objective Description
VULDA-Base	✗	✗	✗	No graph or context integration
VULDA-VCMG	✓	✗	✗	Only apply VCMG simplification
VULDA-LDCA(CFG)	✓	✓	✗	Add control-flow context only
VULDA-LDCA(DDG)	✓	✗	✓	Add data-flow context only
VULDA (Full)	✓	✓	✓	Full model with all components

Analysis and Discussion. Experimental results demonstrate that the VCMG graph structure, serving as a foundational data modeling approach, significantly improves model performance across all datasets. On the SARD dataset, for example, the integration of VCMG results in nearly a 10% increase in accuracy. This improvement is attributed to its ability to reduce redundant nodes in the original code graph, thereby enhancing the compactness and clarity of semantic representation. Furthermore, the VWS module increases the model’s sensitivity to vulnerability-relevant code regions by amplifying the representation of lines deemed critical based on static and structural characteristics. The addition of the LDCA module further strengthens the model’s ability to capture complex semantic dependencies in code. Specifically, control flow aggregation (CFG) and data dependency aggregation (DDG) extract distinct types of contextual information. Each submodule independently contributes to performance improvement; however, removing either CFG or DDG consistently results in lower F1 scores compared to the complete model. This suggests that the two components are complementary in capturing semantic patterns relevant to vulnerabilities. The best performance is achieved when both submodules are combined, highlighting the importance of jointly modeling control-flow and data-dependency contexts for robust vulnerability detection.

Table 5. Ablation Results on VCMG and LDCA Modules in VULDA

Dataset	Methods	Accuracy	Precision	Recall	F1 score
SARD	VULDA-Base	86.14	91.73	88.82	90.25
	VULDA-VCMG	95.71	93.41	96.77	94.86
	VULDA-LDCA(CFG)	97.23	95.78	97.55	96.61
	VULDA-LDCA(DDG)	90.05	89.95	84.62	86.76
	VULDA (Full Model)	97.29	95.79	97.73	96.69
Reveal	VULDA-Base	85.71	56.52	56	56.24
	VULDA-VCMG	90.34	64.47	51.89	51.52
	VULDA-LDCA(CFG)	90.75	72.06	61.58	64.63
	VULDA-LDCA(DDG)	89.71	60.33	52.56	52.9
	VULDA (Full Model)	91.94	81.25	61.91	66.22
FFmpeg+Qemu	VULDA-Base	49.79	48.78	48.85	48.46
	VULDA-VCMG	51.06	49.71	49.74	48.79
	VULDA-LDCA(CFG)	53.02	51.65	51.34	49.64
	VULDA-LDCA(DDG)	53.53	52.67	52.48	52.02
	VULDA (Full Model)	58.84	59.03	57.1	55.55

Ablation Study on the Vulnerability Weight Score (VWS) Module.

The Vulnerability Weight Score (VWS) module provides node-level saliency guidance during the graph neural network aggregation process, thereby enhancing the representation of nodes potentially linked to vulnerabilities. VWS integrates two categories of information sources: (1) static heuristic features, and (2) structural statistical metrics.

To evaluate the individual contributions of these feature types, we construct two variants: one using only static features (VWS_StaticOnly) and another using only structural features (VWS_StructureOnly). We compare them against the full VWS model and a version without any weighting mechanism (VULDA w/o VWS). The experimental configurations are listed in Table 6, and the corresponding results are presented in Table 7.

Table 6. Ablation Settings for Evaluating the VWS Module

Experiment	Static Features	Structural Features	Objective Description
VULDA (w/o VWS)	✗	✗	No weighting features used
VWS_StaticOnly	✓	✗	Use static features only
VWS_StructureOnly	✗	✓	Use structural features only
VULDA (Full Model)	✓	✓	Use both static and structural features

Table 7. Ablation Results on the Impact of the VWS Module

Dataset	Methods	Accuracy	Precision	Recall	F1 score
SARD	VULDA (w/o VWS)	97.23	94.05	97.23	95.54
	VWS_StaticOnly	94.46	91.81	95.67	93.41
	VWS_StructureOnly	97.14	95.64	97.49	96.51
	VULDA (Full Model)	97.29	95.79	97.73	96.69
Reveal	VULDA (w/o VWS)	90.95	73.75	58.46	61.44
	VWS_StaticOnly	90.39	67.89	54.18	55.34
	VWS_StructureOnly	91.49	75.12	70.37	72.41
	VULDA (Full Model)	91.94	81.25	61.91	66.22
FFmpeg+Qemu	VULDA (w/o VWS)	55.4	54.66	54.25	53.7
	VWS_StaticOnly	57.74	57.3	57.14	57.1
	VWS_StructureOnly	57.92	57.55	57.38	57.26
	VULDA (Full Model)	58.84	59.03	57.1	55.55

Analysis and Discussion: With the introduction of the LDCA module, the model already demonstrates strong capability in modeling vulnerability semantics. The VWS module further enhances the sensitivity of the graph neural network during the node aggregation phase. Ablation experiments reveal that using only heuristic static features or structural features individually results in performance bottlenecks. This is because the importance of vulnerable nodes is typically multidimensional, and a single type of feature cannot fully capture their actual role in the code context. In contrast, the complete VWS module, which integrates both static and structural information, more effectively highlights nodes related to potential vulnerabilities. This strengthens the model’s focus on critical paths, leading to an overall performance improvement. These results validate the rationality and necessity of the VWS design.

5.3 Comparative Analysis with Existing Methods

To comprehensively evaluate the practical performance of VULDA, we compare it with several mainstream vulnerability detection approaches, including sequence-based models such as VulDeePecker [2] and SySeVR [9], graph neural network-based models like Devign [3], the large language model-based approach GRACE [22], and the vulnerability image detection method VulCNN [33]. These methods represent the current mainstream technical paradigms in software vulnerability detection. The experimental results presented in Table 8 demonstrate VULDA’s superior detection capability.

Analysis and Discussion:

- **SARD Dataset [31]:** outperforms all other methods across every evaluation metric, indicating its superior capability in modeling vulnerability semantics

Table 8. Performance Comparison of Vulnerability Detection Methods Across Different Datasets. Best results for each metric are highlighted in bold. Darker shading indicates better performance across all methods.

Dataset	Methods	Accuracy	Precision	Recall	F1 score
SARD	VulDeePecker	78.55	78.11	66.74	72.21
	SySeVR	88.25	89.10	83.50	86.13
	Devign	89.06	93.56	91.45	92.49
	VulCNN	89.75	93.10	92.98	93.04
	GRACE	93.54	96.74	95.10	95.90
	VULDA	97.29	95.79	97.73	96.69
Reveal	VulDeePecker	76.37	21.13	13.10	16.17
	SySeVR	74.33	40.07	24.94	30.74
	Devign	87.49	31.55	36.65	36.65
	VulCNN	88.20	35.57	31.83	33.60
	GRACE	89.73	33.21	61.53	43.13
	VULDA	91.94	81.25	61.91	66.22
FFmpeg+Qemu	VulDeePecker	49.91	46.05	32.55	38.14
	SySeVR	47.85	46.06	58.81	51.66
	Devign	56.89	52.50	64.67	57.95
	VulCNN	55.74	58.70	59.92	59.31
	GRACE	59.78	53.94	82.13	65.11
	VULDA	58.84	59.03	57.10	55.55

- when handling labeled vulnerable code snippets. This advantage is largely attributed to the combined effect of the LDCA aggregation mechanism and the VWS weighting strategy.
- **Reveal Dataset [32]:** achieves a significant improvement in Precision (81.25%), far surpassing other methods. Compared with Devign [3] and VulCNN [33], it also demonstrates a better balance in Recall, suggesting that VULDA generalizes well even in complex real-world vulnerability scenarios.
 - **FFmpeg+Qemu Dataset [3]:** dataset contains a large amount of real project code and poses greater challenges. VULDA performs comparably to GRACE [22], with a clear advantage in Precision (59.03%). However, its Recall is slightly lower, indicating room for improvement under high-noise conditions. Future work could incorporate dynamic analysis to further enhance the model’s contextual understanding.

Overall, VULDA’s comprehensive performance across multiple benchmark datasets confirms the effectiveness of its design. Its modular architecture (VCMG + LDCA) not only ensures model interpretability, but also significantly improves detection performance, demonstrating strong practical applicability and scalability.

6 Discussion

Although VULDA demonstrates strong performance across multiple datasets, there remain areas for improvement:

1. **Enhancing VCMG representation:** Currently, the VCMG is constructed based on line-level code information. Future work could incorporate token-level features and line-level vulnerability labels to improve fine-grained semantic modeling and simplify data preprocessing for subsequent research.
2. **Improving context weighting adaptability:** The existing static weight allocation performs well in various scenarios but has limited adaptability to diverse vulnerability patterns. Exploring adaptive attention mechanisms may enhance the model’s sensitivity to critical contextual information.
3. **Classifier architecture optimization:** The current framework uses generic classifiers. Future work could explore specialized architectures and optimization strategies to better detect complex vulnerabilities.

7 Conclusion

This paper presents **VULDA**, a novel vulnerability detection framework that combines the Vulnerability-aware Code Mapping Graph (VCMG) and Local Dependency Context Aggregation (LDCA) to improve semantic modeling and detection accuracy. VCMG maps source code into line-level nodes, emphasizing potential vulnerability areas through structural and static heuristic features. LDCA aggregates semantic contexts from control flow (CFG) and data dependency (DDG) paths to capture complex vulnerability logic. Experiments on the SARD, Reveal, and FFmpeg+Qemu datasets demonstrate that VULDA outperforms existing methods such as VulDeePecker, SySeVR, Devign, VulCNN, and GRACE, achieving up to a 23.09% F1 score improvement on the Reveal dataset. Ablation studies further validate the effectiveness and synergy of each module. VULDA offers a clear and scalable framework with strong practical value and lays the groundwork for future research in fine-grained, context-aware vulnerability detection.

Acknowledgment. This work was supported by the National Natural Science Foundation of China under Grant 62203337. The authors would like to express their gratitude to the creators of the FFmpeg+Qemu, Reveal, and SARD datasets and the open-source code provided by the authors of various open-source projects.

References

1. Shin, Y., Meneely, A., Williams, L., Osborne, J.A.: Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities. *IEEE Trans. Softw. Eng.* **37**(6), 772–787 (2011)
2. Li, Z., et al.: Vuldeepecker: a deep learning-based system for vulnerability detection. *arXiv preprint arXiv:1801.01681* (2018)
3. Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *arXiv Software, Engineering* (2019)
4. Johnson, B., et al.: Why don't software developers use static analysis tools to find bugs? In: *Proceedings of the 2013 International Conference on Software Engineering (ICSE)*, pp. 672–681 (2013)
5. Nguyen, A., et al.: Deep static analysis with context-sensitive graph embedding for vulnerability detection. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 747–761 (2020)
6. Allamanis, M., Brockschmidt, M., Khademi, M.: Learning to represent programs with graphs. *arXiv preprint arXiv:1711.00061* (2018)
7. Guo, D., et al.: Graphcodebert: Pre-training code representations with data flow. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3965–3974 (2020)
8. Wen, X.-C., Chen, Y., Gao, C., Zhang, H., Zhang, J., Liao, Q.: Vulnerability detection with graph simplification and enhanced graph representation learning. In: *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023)*, pp. 2275–2286. *IEEE* (2023)
9. Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z.: Sysevr: a framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secure Comput.* **19**(4), 2244–2258 (2021). <https://doi.org/10.1109/tsec.2021.3051525>
10. Fan, W., Xu, S., Li, X., Feng, B., Peng, Y.: A survey on ai-based software vulnerability detection. *ACM Comput. Surv. (CSUR)* **53**(6), 1–38 (2020)
11. C. Ltd. Checkmarx sast: Static application security testing. Commercial software (2021). <https://www.checkmarx.com/>
12. Wheeler, D.A.: Flawfinder: A static analysis tool for c/c++. Open-source software (2021). <https://www.dwheeler.com/flawfinder/>
13. Secure Software, I.: Rough auditing tool for security (rats). Open-source tool (2021). <https://code.google.com/archive/p/roughauditing-tool-for-security/>
14. Zimmermann, T., Nagappan, N., Williams, L.: Searching for a needle in a haystack: predicting security vulnerabilities for windows vista. In: *Proceedings of the International Conference on Software Testing, Verification and Validation*, pp. 421–428 (2010)
15. Walden, J., Stuckman, J., Scandariato, R.: Predicting vulnerable components: software metrics vs text mining. In: *International Symposium on Software Reliability Engineering*, pp. 23–33 (2014)
16. Perl, H., Dechand, S., Smith, M., et al.: Vccfinder: finding potential vulnerabilities in open-source projects to assist code audits. In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, Denver, Colorado, USA, pp. 426–437 (2015)
17. Meneely, A., Srinivasan, H., Musa, A., Tejeda, A.R., Mokary, M., Spates, B.: When a patch goes bad: exploring the properties of vulnerability-contributing commits. In: *2013 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, pp. 65–74 (2013)

18. Morrison, P., Herzig, K., Murphy, B., Williams, L.: Challenges with applying vulnerability prediction models. In: Proceedings of the 2015 Symposium and Bootcamp on the Science of Security, pp. 1–9 (2015)
19. Younis, A., Malaiya, Y., Anderson, C., Ray, I.: To fear or not to fear that is the question: code characteristics of a vulnerable function with an existing exploit. In: Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy, pp. 97–104 (2016)
20. Shin, Y., Williams, L.: Can traditional fault prediction models be used for vulnerability prediction. *Empir. Softw. Eng.* **18**(1), 25–59 (2013)
21. Li, Z., Zou, D., Xu, S., Chen, Z., Zhu, Y., Jin, H.: Vuldelocator: a deep learning-based fine-grained vulnerability detector. *IEEE Trans. Dependable Secure Comput.* **19**(4), 2821–2837 (2021)
22. Lu, G., Ju, X., Chen, X., Pei, W., Cai, Z.: GRACE: empowering LLM-based software vulnerability detection with graph structure and in-context learning. *J. Syst. Softw.* **212**, 112031 (2024)
23. Li, Q., Han, X.-M., Wu, X.: Deeper insights into graph convolutional networks for semi-supervised learning. In: Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI), pp. 3538–3545 (2018)
24. Pagliardini, M., Gupta, P., Jaggi, M.: Unsupervised learning of sentence embeddings using compositional n-gram features. arXiv preprint [arXiv:1703.02507](https://arxiv.org/abs/1703.02507) (2017)
25. Yamaguchi, F., Golde, N., Arp, D., Rieck, K.: Modeling and discovering vulnerabilities with code property graphs. In: 2014 IEEE Symposium on Security and Privacy, pp. 590–604. IEEE (2014)
26. Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Comput.* **9**(8), 1735–1780 (1997). <https://doi.org/10.1162/neco.1997.9.8.1735>
27. Graves, A., Schmidhuber, J.: Framewise phoneme classification with bidirectional LSTM and other neural network architectures. *Neural Netw.* **18**(5–6), 602–610 (2005). <https://doi.org/10.1016/j.neunet.2005.06.042>
28. Cho, K., et al.: Learning phrase representations using rnn encoder-decoder for statistical machine translation. In: EMNLP, Association for Computational Linguistics, pp. 1724–1734 (2014). <https://doi.org/10.3115/v1/D14-1179>
29. Bahdanau, D., Cho, K., Bengio, Y.: Neural machine translation by jointly learning to align and translate. In: 3rd International Conference on Learning Representations (ICLR 2015), ICLR, San Diego, CA, USA, pp. 1–15 (2015). arXiv preprint [arXiv:1409.0473](https://arxiv.org/abs/1409.0473). <https://doi.org/10.48550/arXiv.1409.0473>
30. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proc. IEEE* **86**(11), 2278–2324 (1998). <https://doi.org/10.1109/5.726791>
31. Software assurance reference dataset. NIST Standard Reference Dataset (2018). <https://samate.nist.gov/SRD/index.php>
32. Chakraborty, S., Krishna, R., Ding, Y., Ray, B.: Deep learning based vulnerability detection: are we there yet?. *IEEE Trans. Softw. Eng.* 3280–3296 (2022). <https://doi.org/10.1109/tse.2021.3087402>
33. Wu, Y., Zou, D., Dou, S., Yang, W., Xu, D., Jin, H.: Vulcnn: an image-inspired scalable vulnerability detection system. In: Proceedings of the 44th International Conference on Software Engineering, pp. 2365–2376 (2022)
34. Kohavi, R.: A study of cross-validation and bootstrap for accuracy estimation and model selection. In: International Joint Conference on Artificial Intelligence (IJCAI), vol. 14, pp. 1137–1143 (1995)
35. Chollet, F.: Keras: the python deep learning library. GitHub repository (2015). <https://github.com/fchollet/keras>

36. Paszke, A., et al.: Pytorch: an imperative style, high-performance deep learning library. In: NeurIPS 2019 Conference Paper (2019). <https://pytorch.org/>. <https://papers.nips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
37. Kingma, D.P., Ba, J.: Adam: a method for stochastic optimization. In: International Conference on Learning Representations (ICLR) (2015). <https://arxiv.org/abs/1412.6980>